AUGMENTED DECISION · FINAL PROJECT REPORT

Flagging bankruptcy *before* it happens

Cost-sensitive screening of Taiwanese firms with LASSO selection and a decision tree

Yassine ERRAJI · **William PELTIER** · **Ben DAESH** · **Simon AMAR**

23 October 2026 · Code: github.com/yassineerraji/Augmented-Decision-Final-Project

ABSTRACT

A credit analyst must decide whether to flag a listed firm as high-risk using its financial ratios. Bankruptcies are rare (3.2 % of 6,819 Taiwanese firms) and a missed one is assumed to cost ten false alarms, so models are judged on the *average misclassification cost per firm*, and a firm is flagged when its estimated bankruptcy probability reaches the Bayes threshold $1/11$. We select variables with an L1-penalised (LASSO) logistic regression, refit inside every cross-validation fold, and feed the 43 selected ratios to a depth-4 decision tree. On 1,364 held-out firms the final model costs **0.179** per firm against **0.323** for not screening (-45 %; 95 % bootstrap interval of the gain $[0.065, 0.228]$), catching 32 of the 44 bankruptcies. The tree uses only 8 ratios and reads as a handful of rules; a random forest would be about 10 % cheaper in cross-validation, the price we pay for interpretability.

1 Problem formulation

Decision and target. A credit analyst reviews a listed firm and chooses between two actions: *flag* it as high-risk (deny or reprice credit, open a closer review) or *do not flag* it. The target is $Y =$ `Bankrupt?`, equal to 1 if the firm went bankrupt under the Taiwan Stock Exchange rules. The inputs x are 95 financial ratios, assumed to be known before the outcome.

Loss and decision rule. The two possible errors do not cost the same (Table 1). With $p = P(Y = 1 \mid x)$, flagging has expected cost $C_{FP}(1 - p)$ and not flagging $C_{FN}\,p$, so the Bayes decision rule (Lecture 1) flags a firm when

$$p \geq \frac{C_{FP}}{C_{FP} + C_{FN}} = \frac{1}{11} \approx 0.09. \tag{1}$$

The costs $C_{FN} = 10$ and $C_{FP} = 1$ are stipulated, not estimated; Section 3.4 shows how the decisions change with the cost ratio.

Table 1. Loss table of the decision problem.

	Healthy firm ($Y = 0$)	Bankrupt firm ($Y = 1$)
Do not flag ($a = 0$)	0	$C_{FN} = 10$ (missed bankruptcy)
Flag ($a = 1$)	$C_{FP} = 1$ (false alarm)	0

Metric and baseline. A model is scored by its empirical risk under the loss table, the *average cost per firm* (lower is better), complemented by recall and precision:

$$\hat{R}_n = \frac{1}{n}\sum_{i=1}^{n} L(\hat{a}_i, y_i) = \frac{C_{FN}\cdot \mathrm{FN} + C_{FP}\cdot \mathrm{FP}}{n}. \tag{2}$$

Accuracy is unsuitable: the majority-class rule "never flag" is 96.8 % accurate while catching no bankruptcy. That rule is our **baseline**; its cost is $10 \times 0.032 \approx 0.323$ per firm.

Data. The data come from the Taiwan Economic Journal for 1999–2009 and are distributed by the UCI Machine Learning Repository [1, 2]. Table 2 summarises them. Two features of the data shape our method. First, the ratios are highly redundant: 50 pairs have a Spearman correlation $|\rho| > 0.95$, involving 47 of the 94 usable ratios (Figure 1). Second, 24 ratios mix values in $[0, 1]$ with values between 6×10^5 and 10^{10}, with nothing in between, which looks like an encoding artefact. The strongest single predictors are profitability ratios (EPS, net income to total assets, ROA), each with a univariate AUC between 0.86 and 0.89. This exploratory analysis is descriptive only; every modelling choice below is made on the training set.

Table 2. The dataset at a glance.

Observations	6,819 listed Taiwanese firms, 1999–2009
Variables	95 financial ratios (one, `Net Income Flag`, is constant and is dropped: 94 remain)
Target distribution	220 bankrupt firms (3.2 %), 6,599 healthy firms
Missing values, duplicates	none, none
Preprocessing required	drop the constant column; handle 24 ratios with extreme values (Section 2.3)

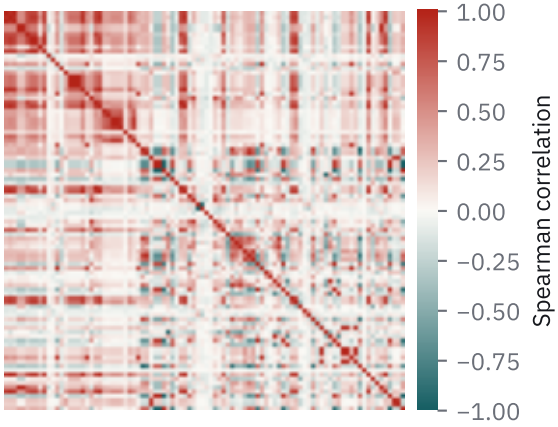


Figure 1. Spearman correlation matrix of the 94 ratios, in dataset column order. Blocks of near-duplicates (profitability, per-share values, growth, leverage, cash flow) motivate variable selection.

2 Methods and choices

2.1 Overview

Figure 2 summarises the approach ("variable selection with decision-tree classification"). A LASSO logistic regression chooses a subset of ratios; a decision tree, grown on the raw values

of those ratios, estimates the bankruptcy probability in each leaf; the Bayes rule (1) turns that probability into a decision.

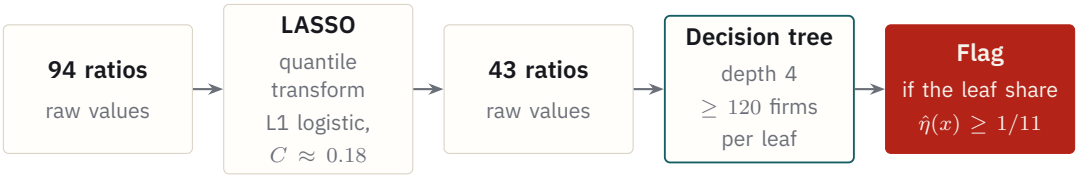


Figure 2. The final pipeline. Everything left of the decision is refit inside every cross-validation fold.

2.2 Data split and validation protocol

We hold out a stratified 20 % test set (1,364 firms, 44 bankrupt) with a fixed seed; the remaining 5,455 firms (176 bankrupt) form the training set. The test set is opened once, at the end, for the frozen final model and the baseline only (Lecture 2). All tuning and model comparisons use the *same* stratified 5-fold cross-validation on the training set. Preprocessing and selection are part of scikit-learn pipelines [3], so in every fold the quantile transform, the LASSO and the tree are refit on four folds and scored on the fifth: no information leaks from the validation fold. Code 1 shows the metric used throughout.

Code 1. The decision cost as a scikit-learn scorer (notebook, Section 3.1).

```
def avg_cost(y_true, y_pred, c_fn=C_FN, c_fp=C_FP):
    return np.mean(c_fn * ((y_true == 1) & (y_pred == 0))
                   + c_fp * ((y_true == 0) & (y_pred == 1)))

cost_scorer = make_scorer(avg_cost, greater_is_better=False)
```

2.3 Preprocessing

The constant column is dropped. The 24 ratios with extreme values are *not* capped: among the 4,816 training firms whose `Total Asset Growth Rate` exceeds 1, the value alone still ranks bankruptcies with an AUC of 0.72, so the extreme values carry signal. The two models need different treatments. The tree only uses the order of the values and receives the raw ratios. The LASSO is sensitive to scale, so it receives a rank-based quantile transform to a standard normal, fitted on the training folds only: it keeps the order of the values, removes the magnitude problem and puts all ratios on a common scale, which a fair L1 penalty requires.

2.4 Variable selection: LASSO logistic regression

For a binary outcome, the LASSO of Lecture 4 becomes an L1-penalised logistic regression with an unpenalised intercept:

$$\min_{b_0,\,\beta} \; -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \log p_i + (1-y_i)\log(1-p_i)\Big] + \lambda\|\beta\|_1, \qquad p_i = \frac{1}{1+e^{-(b_0+x_i^{\top}\beta)}}. \tag{3}$$

The L1 penalty sets many coefficients exactly to zero; the corresponding ratios are dropped. In scikit-learn the penalty is set through $C = 1/(n\lambda)$. We prefer the LASSO to forward selection (Lecture 3.2) because a single parameter controls the number of variables, it copes with

the redundant ratios, and one fit takes under a second, so it can be refit inside every fold; forward selection would need up to 94 fits per added variable, repeated in every fold. The intercept is kept unpenalised as in the course (`intercept_scaling=100` makes `liblinear`'s intercept penalty negligible), and no class weights are used: the LASSO only chooses variables, and the costs enter at the decision stage.

Penalty. C is chosen by 5-fold cross-validation on the training set, minimising the log-loss, a proper scoring rule and the classification counterpart of the validation MSE used in the course, over 13 values from 10^{-3} to 1 (Code 2). The minimum, $C \approx 0.178$, keeps **43 of the 94 ratios**, with a CV log-loss of 0.0855 against 0.143 with no variable (Figure 3). The curve is flat: every C from 0.056 (21 ratios) to 0.562 (59 ratios) is within one standard error of the minimum.

Code 2. LASSO pipeline and choice of the penalty (notebook, Sections 2.1–2.2).

```
lasso = make_pipeline(QuantileTransformer(output_distribution="normal"),
                      LogisticRegression(l1_ratio=1, solver="liblinear",
                                         intercept_scaling=100, random_state=SEED))
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
C_grid = np.logspace(-3, 0, 13)
lasso_cv = GridSearchCV(lasso, {"logisticregression__C": C_grid}, cv=cv,
                        scoring="neg_log_loss", n_jobs=-1).fit(X_train, y_train)
```

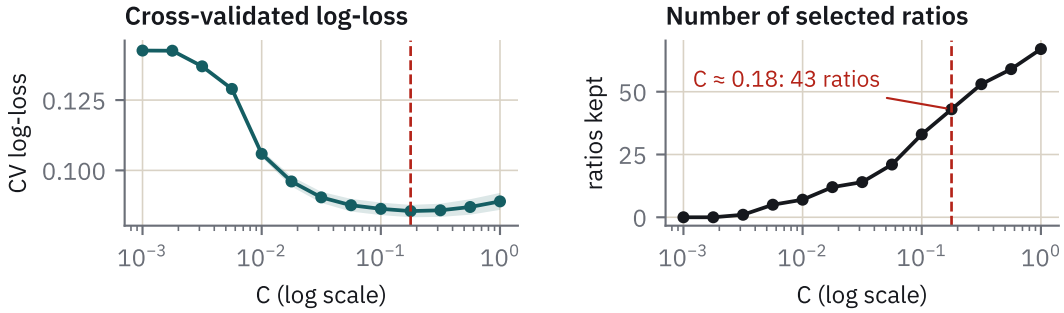


Figure 3. Choice of the LASSO penalty. Left: cross-validated log-loss (mean $\pm$ 1 SE). Right: number of ratios kept when refit on the full training set. Dashed line: chosen $C \approx 0.18$.

What is selected, and how stable it is. The leading coefficients (Table 3) cover leverage, profitability and liquidity, with intuitive signs: more debt raises the predicted risk, more profit and liquidity lower it. Following Lecture 4, we refit the whole procedure on 100 stratified bootstrap resamples of the training set (Figure 4). Only 15 of the 43 selected ratios are kept in at least 80 % of the resamples, and some ratios left out are often selected, such as the R&D expense rate (81 %): correlated ratios swap places. The selected set is a good predictive shortlist, not a list of causes.

2.5 Decision tree and decision rule

A classification tree (CART, Gini impurity $G(q) = 2q(1-q)$, Lecture 3) is grown on the raw values of the selected ratios. The share of bankrupt training firms in a leaf estimates $\hat{\eta}(x) = P(Y = 1 \mid x)$ for every firm reaching it, and the firm is flagged when $\hat{\eta}(x) \geq 1/11$. The costs therefore enter through the *threshold*, not through class weights: this keeps the leaves' reading as probabilities

Table 3. The eight largest LASSO coefficients at $C \approx 0.178$ (quantile-normal scale, so magnitudes are comparable).

Ratio	Coefficient	Ratio	Coefficient
Total debt / total net worth	$+0.819$	Quick ratio	-0.320
ROA(B) before interest	-0.477	Working capital turnover	$+0.314$
Retained earnings / total assets	-0.415	Persistent EPS (4 seasons)	-0.311
Interest-bearing debt interest rate	$+0.350$	Borrowing dependency	$+0.308$

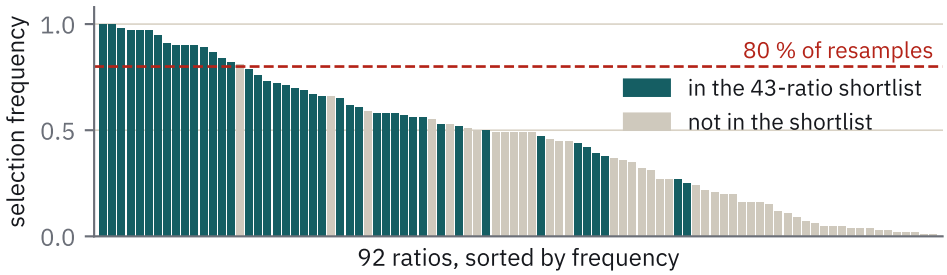


Figure 4. Selection stability: share of 100 bootstrap refits in which each ratio is selected (92 ratios selected at least once). Teal: in the 43-ratio shortlist; grey: not in it.

and lets us test other cost ratios without retraining. In code, the whole pipeline is wrapped in a `FixedThresholdClassifier`, so its predictions are directly the decisions (Code 3).

Hyperparameters. We tune the maximum depth, $\{2, 3, 4, 5, 6, 8, \text{no limit}\}$, and the minimum number of firms per leaf, $\{1, 10, 20, 40, 80, 120, 160, 200\}$, on the decision cost. Large leaves are needed for the leaf shares to be reliable probabilities: with a 3.2 % bankruptcy rate, a leaf of 120 firms holds only about four bankruptcies. Figure 5 shows the effect. With leaves of one firm, the training cost falls to zero as the tree deepens while the CV cost climbs from 0.17 to 0.23: the tree memorises the training firms (Lecture 2). With at least 120 firms per leaf, the gap stays small and depth 4 is best (0.151).

Code 3. Final pipeline M2 and its tuning (notebook, Sections 2.4 and 3.3).

```
selector = SelectFromModel(lasso.set_params(logisticregression__C=best_C),
                           importance_getter="named_steps.logisticregression.coef_",
                           threshold=1e-5)  # keep the non-zero coefficients
pipe = FixedThresholdClassifier(
    Pipeline([("select", selector),
              ("tree", DecisionTreeClassifier(random_state=SEED))]),
    threshold=THRESHOLD)                    # flag when P(bankrupt | x) >= 1/11
tree_grid = {"max_depth": [2, 3, 4, 5, 6, 8, None],
             "min_samples_leaf": [1, 10, 20, 40, 80, 120, 160, 200]}
m2_grid = {"estimator__select__estimator__logisticregression__C": [best_C],
           **{f"estimator__tree__{k}": v for k, v in tree_grid.items()}}
m2 = GridSearchCV(pipe, m2_grid, cv=cv, scoring=cost_scorer, n_jobs=-1,
                  return_train_score=True).fit(X_train, y_train)
```

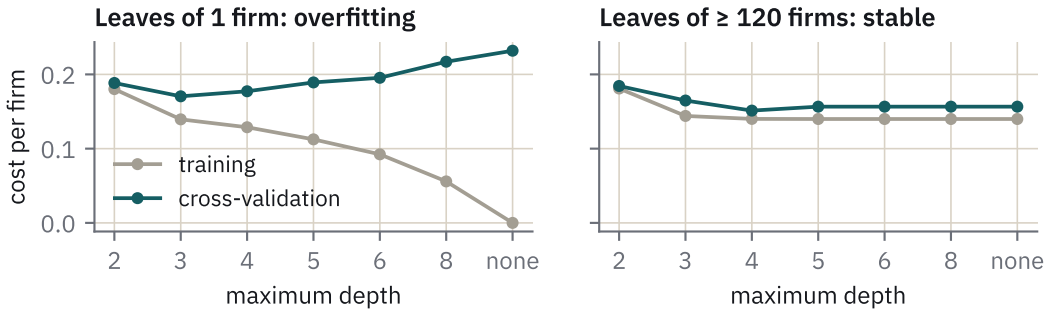


Figure 5. Validation curves of the final pipeline: training and cross-validated cost per firm against the maximum depth, for leaves of one firm (left) and of at least 120 firms (right).

3 Evaluation

3.1 Cross-validated model selection

We compare four candidates on the same folds: **M0**, the baseline; **M1**, a tuned tree on all 94 ratios; **M2**, the LASSO selector with C fixed at its Section 2.4 value, followed by a tuned tree; and **M3**, the same pipeline with C tuned jointly with the tree over the seven grid values from 0.01 to 0.316. The selection rule was fixed before seeing these results: M2, the course protocol, is the reference, and an alternative replaces it only if its CV cost is lower by more than one standard error. Because all candidates share the folds, the comparison is *paired*: for each fold k we compute the gain $\Delta_k = \text{cost}_k(\text{M2}) - \text{cost}_k(\text{alternative})$ and its standard error $s_\Delta/\sqrt{5}$ (Code 4).

Table 4. Cross-validated results (5 folds, training set). Paired gain of each alternative over M2: mean $\pm$ standard error, and number of folds in which it is cheaper than M2.

Candidate	CV cost	Depth	Leaf	C	Ratios	Paired gain over M2
M0 Never flag	0.323	–	–	–	0	-0.171 ± 0.009 (0/5)
M1 Tree, all ratios	0.163	4	120	–	94	-0.012 ± 0.005 (0/5)
M2 LASSO + tree	**0.151**	4	120	0.178	43	reference
M3 LASSO (joint C) + tree	0.148	3	10	0.01	7	$+0.003 \pm 0.007$ (3/5)
M4 LASSO + random forest	0.136	–	10	0.178	43	$+0.015 \pm 0.007$ (5/5)

Code 4. Paired selection rule (notebook, Section 3.4).

```
# per-fold gain of each candidate over M2 (> 0: cheaper than M2)
gains = fold_costs.rsub(fold_costs["M2 LASSO, C fixed"], axis=0)
rule = pd.DataFrame({"mean gain over M2": gains.mean(),
                     "paired SE": gains.std() / np.sqrt(5),
                     "folds cheaper than M2": (gains > 0).sum()})
rule["replaces M2"] = rule["mean gain over M2"] > rule["paired SE"]
```

Table 4 and Figure 6 give the results. Every tree halves the cost of the baseline. **Variable selection helps**: M2 beats the tree on all 94 ratios by 0.012 ± 0.005 and is cheaper or tied on every fold. Joint tuning (M3) is not significantly better (0.003 ± 0.007), although it shows that a 7-ratio selection works about as well. M2 is therefore the final model.

Benchmark: the price of interpretability. To measure what a single readable tree costs, we also cross-validated a random forest of 300 trees on the LASSO-selected ratios (M4), tuning its leaf size. It is a benchmark, not a candidate: the brief asks for a decision-tree classifier and our recommendation relies on reading the tree's rules, so M4 is never scored on the test set. It is about 10 % cheaper (0.015 ± 0.007, cheaper on all five folds), because averaging 300 trees gives much finer probabilities than nine leaves.

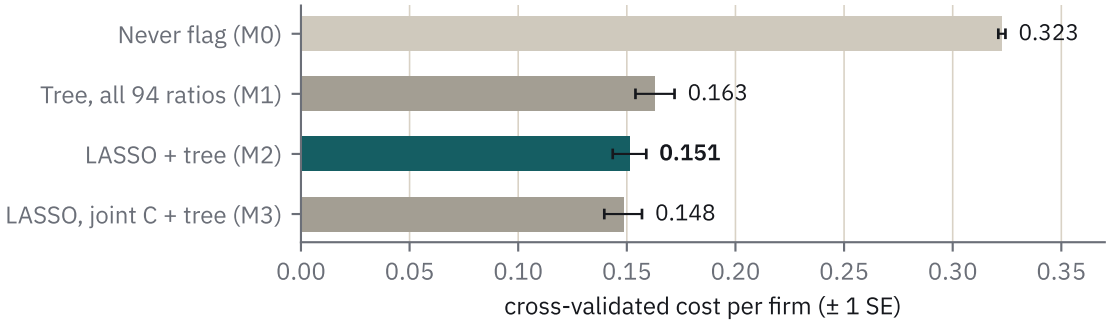


Figure 6. Cross-validated cost per firm of the four candidates ($\pm$ 1 standard error across folds).

3.2 The final model

Refit on the full training set, the LASSO keeps 43 ratios and the tree, of depth 4, has **9 leaves and uses only 8 of them** (Figure 7): the tree performs a second, finer selection.

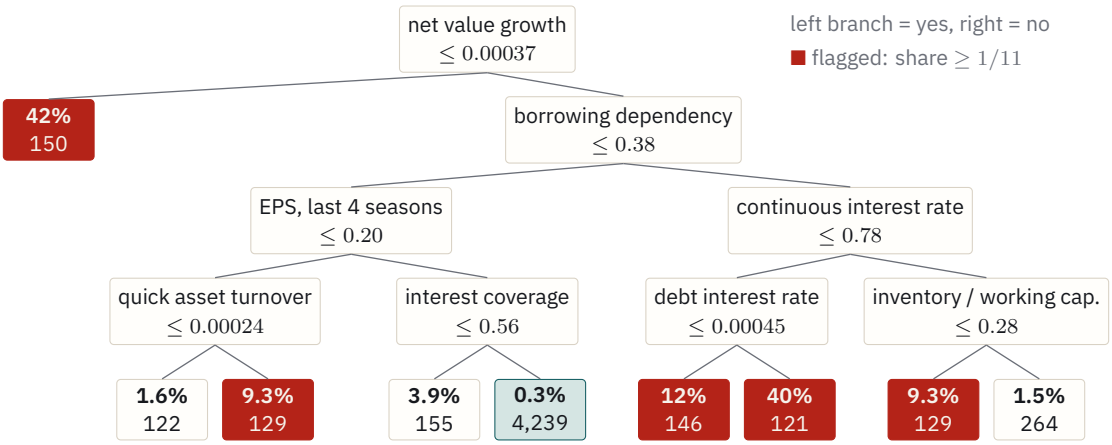


Figure 7. The final tree. Each leaf shows its share of bankrupt training firms and its number of firms; red leaves are flagged.

3.3 Test-set performance

The frozen model and the baseline are scored once on the 1,364 test firms (Table 5, Code 5). The model **costs 0.179 per firm against 0.323**, a 45 % reduction. It catches 32 of the 44 bankruptcies (recall 73 %) at the price of 124 false alarms (precision 21 %): 156 firms are flagged, about one in nine. Its accuracy (90 %) is lower than the baseline's (97 %), which confirms that accuracy would have rewarded the useless rule.

Uncertainty. With a loss bounded in $[0, C_{FN}]$ and $m = 1{,}364$ test firms, Lecture 2's Hoeffding bound gives a 95 % half-width of $C_{FN}\sqrt{\log(2/\alpha)/(2m)} = 0.37$, wider than the gap between the two rules: valid, but uninformative. We therefore use a 95 % percentile bootstrap over the test

Table 5. Test-set performance (1,364 firms, 44 bankrupt). Right: confusion matrix of the final model.

	Final model	Never flag
Average cost per firm	**0.179**	0.323
Recall	0.727	0.000
Precision	0.205	–
Accuracy	0.900	0.968
Firms flagged	156	0

	not flagged	flagged
healthy	1,196	124 false alarms
bankrupt	12 missed	32 caught

Code 5. Test predictions and bootstrap of the test costs (notebook, Sections 4.1–4.2).

```
y_pred = final_model.predict(X_test)        # decisions at the 1/11 threshold
y_base = np.zeros(len(y_test), dtype=int)   # "never flag"
boot = [resample(np.arange(len(y_test)), random_state=b) for b in range(1000)]
boot_costs = pd.DataFrame(
    [{"Final model": avg_cost(y_test.values[i], y_pred[i]),
      "Baseline": avg_cost(y_test.values[i], y_base[i])} for i in boot])
```

firms, with the predictions fixed and without stratification, so that the number of bankruptcies varies as in a new sample (Figure 8). The paired difference lies in $[-0.228, -0.065]$, entirely below zero: the model's advantage is not an artefact of this test set. The test cost is higher than the CV estimate (0.151), which nevertheless lies inside the model's interval; the gap is expected, since the test cost is noisy with only 44 bankruptcies and the CV cost of the chosen configuration, the best of 56, is slightly optimistic.

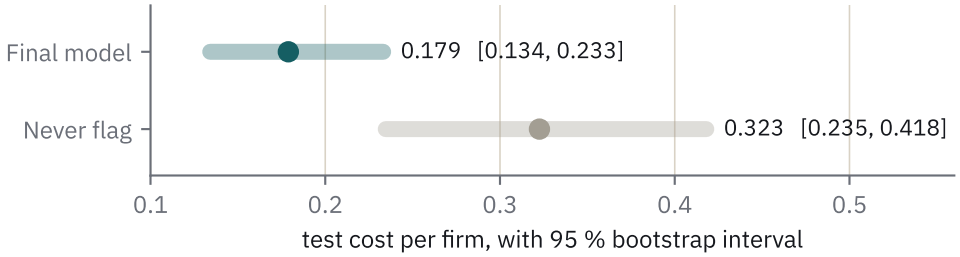


Figure 8. Test cost per firm with 95 % bootstrap intervals (1,000 resamples of the test firms). Gain of the model over the baseline: $[0.065, 0.228]$ per firm.

3.4 Sensitivity to the cost ratio

The 10:1 ratio is an assumption. For a ratio $r = C_{FN}/C_{FP}$ the Bayes threshold becomes $1/(1+r)$; since the tree is not retrained, only the threshold changes (Table 6). The model beats the baseline under every ratio, cutting its cost by 35 %, 45 % and 59 %. Each leaf is flagged once r exceeds its *switching ratio*

$$r^* = \frac{1-\hat{\eta}}{\hat{\eta}}, \qquad \text{since } \hat{\eta} \geq \frac{1}{1+r} \iff r \geq r^*. \tag{4}$$

Figure 9 shows that the two riskiest leaves (42 % and 40 %) are flagged for any ratio above 1.5, while two leaves (9.3 %) switch at 9.75, just below our assumed ratio. No leaf switches between 9.75 and 24.8, which is why ratios 10 and 20 give identical decisions.

Table 6. Test-set results under three cost ratios (tree unchanged, threshold $1/(1+r)$).

Cost ratio r	Threshold	Model cost	Baseline cost	Recall	Precision	Firms flagged
5	0.167	0.106	0.161	0.545	0.353	68
10	**0.091**	**0.179**	**0.323**	**0.727**	**0.205**	**156**
20	0.048	0.267	0.645	0.727	0.205	156

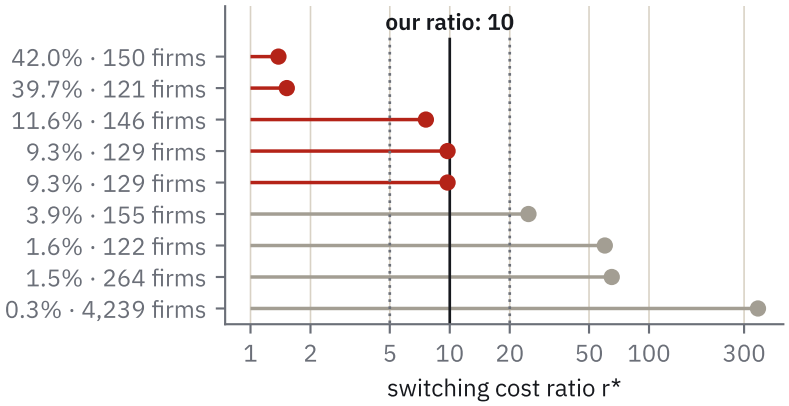


Figure 9. Switching cost ratio of each leaf (training share of bankrupt firms and number of firms). Red: flagged at our ratio of 10; dotted lines: ratios 5 and 20.

4 Evidence and recommendation

4.1 What the tree says

Reading the flagged leaves of Figure 7, the model raises an alarm in three situations: (i) the 2.7 % of firms with the weakest net value growth, of which 42 % went bankrupt; (ii) heavy reliance on borrowing combined with weak profitability, 40 % bankrupt when the debt is expensive and 12 % otherwise; (iii) two narrower pockets at 9.3 %, flagged only just. Conversely, firms with moderate borrowing, EPS above 0.20 and an interest coverage ratio above 0.56 hold 78 % of the training firms and a bankruptcy share of 0.3 %. These are predictive associations, not causal effects.

4.2 Recommendation

Following the decision-memo structure of Lecture 1:

1. **Action and information.** Flag every firm whose leaf has a bankruptcy share of at least $1/11$, using 8 financial ratios read by a depth-4 tree.
2. **Assumptions.** A missed bankruptcy costs ten false alarms, and the leaf shares of 5,455 training firms are taken as bankruptcy probabilities.
3. **Comparison with the alternative.** On unseen firms the model costs 0.179 per firm against 0.323 without screening, catching 32 of 44 bankruptcies for 124 false alarms. A random forest would lower the cross-validated cost by about 10 %, but without the readable rules this recommendation relies on.
4. **Conditions that would change the choice.** With a cost ratio below 9.75, two leaves would no longer be flagged, and below 7.6 a third; any ratio between 9.75 and 24.8 leaves the decisions unchanged.

4.3 Limitations

- **Stipulated costs.** The 10:1 ratio is an assumption, and two leaves depend on it being at least 9.75.
- **No time dimension.** Firms are pooled over 1999–2009 without a year, so the evaluation is a random split, not a forecast of future years; economic conditions may shift (Lecture 2).
- **Possibly repeated firms.** The rows may include several years of the same firm; a random split can then place one firm in both training and test, making the test cost optimistic. Without a firm identifier this cannot be checked.
- **Few bankruptcies.** With 176 bankruptcies in training and 44 in test, CV standard errors are about 0.008, the test interval is wide, and leaf shares rest on 2 to 63 bankruptcies. Nodes 6 and 15 are flagged with a share of 0.093, only 0.002 above the threshold, while that share's standard error is about $\sqrt{0.093 \times 0.907/129} \approx 0.026$.
- **No nested cross-validation.** M2 and M3 were compared on the folds used to tune them; the pre-registered rule limits but does not remove this optimism.
- **Unstable variable list.** Only 15 of the 43 LASSO ratios are robustly selected, and a 7-ratio selection works about as well.
- **Data quality.** Three of the eight ratios used by the tree contain extreme values that look like encoding artefacts; the order-based treatment limits, but may not remove, their impact.

Before deployment, the frozen procedure should be refit on all 6,819 firms and validated on more recent data.

5 Reproducibility

All results come from a single notebook, `src/bankruptcy_prediction.ipynb`, executed top to bottom with fixed seeds (`SEED = 42`); it runs in about 4.5 minutes with Python 3.12 and the pinned versions of `requirements.txt` (scikit-learn 1.9.1, pandas 3.0.6). The notebook stops with an explicit message if scikit-learn is older than 1.8, because earlier versions silently replace the L1 penalty. A script, `slides/make_figures.py`, exports the figures of this report from that same notebook.

```
uv venv --python 3.12 && uv pip install -r requirements.txt
.venv/bin/jupyter nbconvert --to notebook --execute --inplace \
    src/bankruptcy_prediction.ipynb
```

References

[1] UCI Machine Learning Repository (2020). *Taiwanese Bankruptcy Prediction*. doi:10.24432/C5004D.

[2] D. Liang, C.-C. Lu, C.-F. Tsai and G.-A. Shih (2016). Financial ratios and corporate governance indicators in bankruptcy prediction: a comprehensive study. *European Journal of Operational Research*, 252(2), 561–572.

[3] F. Pedregosa et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[4] Augmented Decision, lecture decks 1–4: decision theory, statistical learning, decision trees, variable selection and regularization.

A LLM assistance

As the project brief requires, this appendix discloses how a large language model (LLM) helped prepare this work. It describes the tool and what it supported (A.1), the working protocol the authors imposed on it (A.2), the authors' decisions and modifications (A.3), every prompt the authors sent, verbatim (A.4), and the full content of the protocol and discussion files (A.5).

A.1 Tool and scope

We used **Claude Opus 5.5** (Anthropic) through **Claude Code**, its coding assistant integrated in Visual Studio Code, during a single working session on 22 October 2026. The LLM worked inside the project folder: it read the files (including the lecture decks and the brief), ran code in the project's Python environment, and wrote files. Every part of this report was prepared with the LLM; Table 7 identifies what it supported and what the authors did.

Table 7. Parts of the work supported by the LLM.

Part	Role of the LLM	Role of the authors
The notebook, source of every number, table and figure of this report	Wrote the code and markdown cells following the approved plans, ran the notebook, exported the figures	Wrote the rules, reviewed and edited every plan, chose between the options, reviewed code and recaps, edited cells
Report, Sections 1–5 (text, tables, figures, code snapshots)	Drafted the whole report from the notebook outputs	Specified the content (project brief), reviewed
Appendix A	Assembled from `INSTRUCTIONS.md`, the AgentDiscussion files and the session transcript	Specified its content
Slide deck, README, repository	Built on request	Specified the constraints, reviewed

A.2 Working protocol

Before starting, the authors wrote `INSTRUCTIONS.md` (reproduced in A.5). It fixes three things: the deliverable (one notebook implementing the four steps of this report), **simplicity rules** for the code (one task per cell, minimal code, a comment at the top of every cell, each step building on the previous one, every assumption or decision stated in a short markdown cell, a coherent markdown structure), and a **step-by-step workflow** instead of a one-shot project (Figure 10).

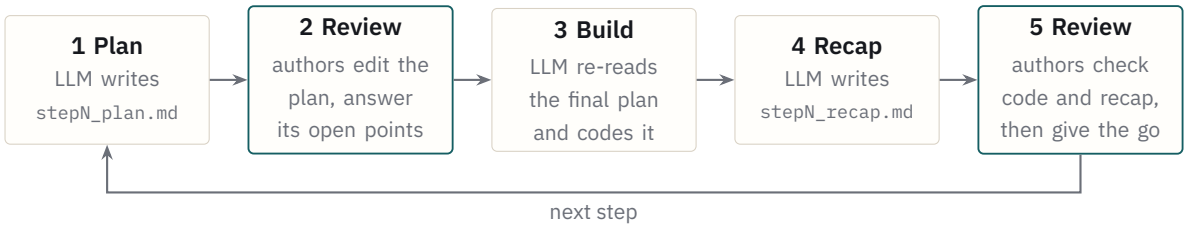


Figure 10. The plan–recap loop of `INSTRUCTIONS.md`, run once per step. Teal: the authors; grey: the LLM.

The written trace of this loop is the `AgentDiscussion` folder, with one sub-folder per step.

A **plan** (`stepN_plan.md`) was written at the start of each step: the LLM's thoughts, assumptions and rationale, the decisions it would record in the notebook, the planned cells, a preview from training-set checks, and a closing list of *points for the authors to confirm or change*. The authors reviewed it, edited it directly when they disagreed (for instance, Step 1, §5), and answered the open points in the conversation; their decisions were then recorded in the plan as "decided". The LLM re-read the plan in its final state before coding. A **recap** (`stepN_recap.md`) was written after coding: what was built, the key results, every difference from the plan, and what carries over to the next step. The next step started only after the authors had reviewed the code and the recap. After Step 4, the authors added a **final review**: they asked for a critical assessment without any modification, then decided on each proposed fix one at a time, after asking what it would bring.

A.3 Authors' decisions and modifications

Table 8 summarises how the authors steered the work; the numbers refer to the prompts of A.4.

Table 8. Main decisions and modifications made by the authors.

Phase (prompts)	Decisions and modifications
Setup (1–2)	Let the LLM create the Python environment; required the notebook to be saved with its outputs; approved the file naming; quality not to be sacrificed to the deadline.
Step 1 (3–5)	Edited the plan to run the exploratory analysis on the full dataset; kept the 10:1 costs and the 20 % test set; asked for a correlation matrix (accepting Spearman over Pearson) and for proper mathematical rendering.
Step 2 (6–10)	Asked to compare the course rule for the LASSO penalty with tuning it jointly with the tree, under a one-standard-error rule; asked for an explanation of the stability plot, which led to a legend and an 80 % line; added their own explanation cell, an imports glossary and a plotting style to the notebook.
Step 3 (11–12)	Followed the recommendations: costs through the threshold, the tuning grid, and the agreed selection rule without a tie-breaker.
Step 4 (13–15)	Restricted the test set to the final model and the baseline; asked for explanations of the bootstrap and of the switching ratios before approving.
Final review (16–33)	Asked for a critical review without modification; applied, after discussing each one, the version check, the repeated-firms limitation, the paired comparison and the leaf-noise limitation; removed every claim whose numbers the notebook did not reproduce, for simplicity; kept the single tree and added the random-forest benchmark.
Organisation	Moved the notebook to `src/`, sorted `AgentDiscussion` into one folder per step, and removed the log of the final review.
Deliverables (34–43)	Requested the README, the push to GitHub with the course material excluded, the slide deck (Arial version kept), this report and this appendix.

A.4 Prompt log

The 43 prompts sent by the authors during the session, verbatim, as typed. Context lines added automatically by the editor (such as the name of the file open in the IDE) are omitted; times are in UTC. The LLM's answers took the form of the plan and recap files of A.5, of the work itself, and of explanations given in the conversation.

#1	2026-10-22 08:34 UTC	
Setup	Hey Claude, take a look at the project and the INSTRUCTIONS.md file and confirm that you understand how we will proceed. If any questions, ask them	
#2	2026-10-22 08:40 UTC	
Setup	1. No environment for now, you can build one. 2. Yes 3. perfect 4. Indeed but let's not sacrifice quality	
#3	2026-10-22 08:51 UTC	
Step 1	Go	
#4	2026-10-22 09:04 UTC	
Step 1	Can we add a correlation matrix to the EDA ?	
#5	2026-10-22 09:09 UTC	
Step 1	One last detail, the markdown rendering for mathematical expression in 1.1 are not great (example : C_FN etc), do you think you could apply actual adapted mathematical markdown for these expressions'	
#6	2026-10-22 09:11 UTC	
Step 2	Excellent, let's move on to part 2.	
#7	2026-10-22 09:30 UTC	
Step 2	Would it be sound to do as in the course (lowest cross-validated log-loss) and compare it with the method of tuning it together with the tree in step 3 ?	
#8	2026-10-22 09:31 UTC	
Step 2	Go	
#9	2026-10-22 09:44 UTC	
Step 2	Can you explain the bootstrap selection frequency plot ? Im not sure to understand it. Explain the two colors	
#10	2026-10-22 09:54 UTC	
Step 2	Yes make that change.	
#11	2026-10-22 09:57 UTC	
Step 3	Excellent, let's move on to the step 3 plan.	
#12	2026-10-22 10:05 UTC	
Step 3	Go, let's follow your recommendation	
#13	2026-10-22 11:21 UTC	
Step 4	Let's move on to the step 4 plan	
#14	2026-10-22 11:28 UTC	
Step 4	1. Only the final model and the baseline 2. explain what you mean 3. explain what you mean	
#15	2026-10-22 11:30 UTC	
Step 4	Go	

#16 2026-10-22 12:04 UTC

Final review Great, now I want you to take a step back, review the full codebase, then the notebook. Do not modify a single thing, I just want you to look at it, be critical, and come up with a fair judgement of what we have done. Are there an pothholes, is it solid and rigorous ?
Evaluate and come back here in the discussion with a report.
Be concize and straight to the point.
If any issues, suggest improvements.

#17 2026-10-22 12:18 UTC

Final review For point 3, is the absent evidence important or can we simply get rid of it ?

#18 2026-10-22 12:19 UTC

Final review As you know, our goal is to make the work the simplest possible, therefore, get rid of everything you can get rid of without destroying the reasoning.

#19 2026-10-22 12:23 UTC

Final review What could we expect from fixing number 2 ?

#20 2026-10-22 12:24 UTC

Final review Go for it

#21 2026-10-22 12:24 UTC

Final review what would fixing #1 look like ?

#22 2026-10-22 12:25 UTC

Final review Go for it

#23 2026-10-22 12:31 UTC

Final review What would we gain from fixing #4 ?

#24 2026-10-22 12:40 UTC

Final review Let’s fix it.

#25 2026-10-22 12:50 UTC

Final review Explain what we gain from fixing #5 ?

#26 2026-10-22 12:56 UTC

Final review Can we fix this while staying as simple as we want to ?

#27 2026-10-22 12:56 UTC

Final review Go

#28 2026-10-22 12:59 UTC

Final review Excellent, now there is one issue I am concerned with, the perfomance of the model. What can we do to improve it ?

#29 2026-10-22 13:03 UTC

Final review Ok let’s [Request interrupted by user]

#30 2026-10-22 13:03 UTC

Final review Let’s go with A.

#31 2026-10-22 13:12 UTC

Final review How many lines of code/comments/Markdown are there in the full notebook ?
Make a short summary table.

#32 2026-10-22 13:17 UTC

Final review What are the results of the random forest model ? compare them to the tree

#33 2026-10-22 13:29 UTC

Final review Do we now have the final version of the notebook ?

#34 2026-10-22 13:38 UTC

Repository Yes let's now build the final README for this project, make it professional and visually beautiful. Do not make it too long.

#35 2026-10-22 13:40 UTC

Repository are these also clickable links ?
![Python](https://img.shields.io/badge/Python-3.12-3776AB?logo=python&logoColor=white)
![scikit-learn](https://img.shields.io/badge/scikit--learn-1.9-F7931E?logo=scikitlearn&logoColor=white)
![pandas](https://img.shields.io/badge/pandas-3.0-150458?logo=pandas&logoColor=white)
![Data](https://img.shields.io/badge/data-UCI%20·%20CC%20BY%204.0-lightgrey)

#36 2026-10-22 13:41 UTC

Repository YES

#37 2026-10-22 13:43 UTC

Repository Let's push the folder to the following repo : git remote add origin https://github.com/yassineerraji/Augmented-Decision-Final-Project.git
Before that, let's create a .gitignore to avoid tracking DS_Store and .venv.

#38 2026-10-22 13:56 UTC

Slide deck Great, now we want to build a slide deck to present for our final presentation.
For context, it should not exceed 15 minutes and should include the essentials of our project.
The names are Yassine ERRAJI, William PELTIER, Ben DAESH, Simon AMAR
We want the slides to be very clean and professional yet very original and not look like the main claude templates. It should stand out.
It should be built in LaTeX and compiled in pdf, obviously following the classic landscape slide deck format.
Include the very main formulas that we build the project on top of (avoid the too complex ones that do not appear in the lecture decks).
Include the most relevant visuals that we obtained from the notebook (make sure that its format is adapted to the slide deck).
Also include schemas or graphs that you create on your own if you believe it would help present better and help people understand better our project.
One Important element, slides must be readable and not too dense, therefore, we would rather have more slides that are more readable and less content heavy that fewer dense ones.
Let's do this inside of a slides folder at the root of the repo.

#39 2026-10-22 15:36 UTC

Slide deck In the slide 25/28, in the head of the plot, the text is not well displayed (our ratio: 10), fix it.

#40 2026-10-22 15:43 UTC

Report Okay great now we want to build a properly compiled PDF report containing:
•Methods and choices. Explain your approach. Motivate the model, preprocessing, and hyperparameters, e.g., tree depth, leaf size, or regularization penalty.
•Evaluation. Compare with a baseline. Tune by validation or cross-validation, fitting preprocessing and selection within each training split. Report final test performance.
•Evidence and recommendation. Include readable code snapshots, labeled plots, and result tables that justify your final model or variable set, recommendation, and limitations.
Build it in a folder called report

#41 2026-10-22 15:57 UTC

Slide deck Now I want you to reproduce another exactly same version of the slide deck we have but this time using the Arial font.

#42 2026-10-22 16:06 UTC

Slide deck Okay excellent, let's delete the previous version and keep the arial one as the main.

#43 2026-10-22 16:37 UTC

Appendix Great, we want to add an appendix to the report about LLM Assistance (take a look at the ProjectRequirements.pdf).
In that section we must explain exactly the procedure of the INSTRUCTIONS.md and AgentDiscussion folder.
Detail the workflow of plan and recap, my modifications and instructions etc..
If possible, include the content of the files in a nice rendering.

A.5 Protocol and discussion files

The files below are reproduced in full, rendered from Markdown, as they stand at the end of the project. Paths written inside them reflect the folder layout at the time (the notebook was later moved from `notebooks/` to `src/`).

`INSTRUCTIONS.md` — Working rules and workflow, written by the authors

We are working on the Taiwanese Bankruptcy Prediction

– (https://archive.ics.uci.edu/dataset/572/taiwanese+bankruptcy+prediction), UCI Machine Learning Repository. Collected from the Taiwan Economic Journal, 1999–2009.

Our goal is to build a 4-step project that goes as follows:

1. Problem Formulation & Exploratory Data Analysis
2. Variable Selection
3. Model Selection and Decision Tree Tuning
4. Evaluation of the final model

Our end-deliverable is one notebook file (ipynb), containing the code that implements the 4 steps. **IMPORTANT**: WE MUST CODE IN THE MOST SIMPLE WAY POSSIBLE. WHAT IS MEANT BY SIMPLE IS:

– A python cell only does one task at a time.
– We always privilege code minimality, what can be done in two lines of codes must be done in no more than two lines, always.
– We always privilege simplicity and correctness over complexity and long code that is hard to visually understand.
– We always include comments in the beginning of a Python cell to explain what is being done.
– All the steps we code must build on top of the previous ones.
– When doing assumptions, or making decisions, they must never be done silently and appear in the form of concise markdowns cells.
– Markdown cells must follow a clean and coherent structure all the way through the notebook.

In the context folder you will find two elements:

1. The lecture decks of the course that will serve as great context to scope well the work we are doing.
2. The official instruction file for the project, provided by our teacher. Feel free to read it to have context on the final submission but don't pay too much attention to it for now as we are only working on the code in a notebook for now. Also, don't pay attention to the LLM disclosure element, I will take care of it when writing the report file.

The workflow will be as follows:

We will be working step by step and not one-shot the full project.

1. You will explain very concisely your thoughts, assumptions and rationale in a folder called AgentDiscussion, where you will add a new markdown file at the beginning of each of the 4 steps of the project.
2. I will review in details the markdown files and either leave them as is or bring modification to them.
3. You will check the last state of the markdown cell and build according to it.
4. After you end coding, write a recap markdown file in the same AgentDiscussion folder and wait for me to review the code and the markdown file before signaling that we can proceed with the next step of the project.

`AgentDiscussion/step1/step1_plan.md` Step 1 plan: written by the LLM, reviewed and edited by the authors

Step 1 — Problem Formulation & EDA (plan)

1. Problem formulation

Element	Proposal
Target	`Bankrupt?` (1 = the firm went bankrupt under Taiwan Stock Exchange rules, 0 = it did not)
Inputs	95 financial ratios, assumed to be known *before* the outcome (so no leakage)
Decision supported	A credit analyst decides whether to **flag** a firm as high-risk (deny or reprice credit, start a closer review) or **not flag** it
Actions	a = 1 flag, a = 0 do not flag

Loss table (Lecture 1). This is an assumption, not an estimate:

	Healthy (Y = 0)	Bankrupt (Y = 1)
Not flag (a = 0)	0	**C_FN = 10** (credit loss)
Flag (a = 1)	**C_FP = 1** (unnecessary review, lost client)	0

- **Bayes decision rule:** flag a firm when P(bankrupt | x) ≥ C_FP / (C_FP + C_FN) = **1/11 ≈ 0.09**.
- **Primary metric:** average misclassification cost per firm, (10·FN + 1·FP) / n. This is the empirical risk under the loss table, so lower is better.
- **Secondary metrics:** the confusion matrix, recall (share of bankruptcies caught) and precision (share of flags that are real bankruptcies).
- **Why not accuracy:** only 3.2% of firms are bankrupt, so "never flag" already reaches 96.8% accuracy.
- **Reference baseline:** "never flag" (the majority class) costs about 10 × 0.032 ≈ **0.32 per firm**. Every model has to beat this.
- **Sensitivity:** in Step 4 we will re-run the evaluation with cost ratios of 5 and 20, because 10 is a stated assumption.

2. Assumptions and decisions (each one becomes a short markdown cell)

1. **The data has no firm ID and no year, so firms are treated as independent draws.** A time-based split is impossible, so we use a random stratified split. This becomes a stated limitation in Step 4.
2. **Drop `Net Income Flag`.** It has the same value in every row, so it carries no information.
3. **Hold out a stratified 80/20 train/test split right after loading** (`random_state = 42`). That gives a test set of about 1,364 firms, about 44 of them bankrupt. The test set stays locked until Step 4.
4. **Run the EDA on the training set only**, so that no later decision uses test information (Lecture 2: "condition on every choice made using training data").
5. **Leave the raw values unchanged in Step 1.** 24 columns mix values in $[0, 1]$ with values between about 6e5 and 1e10. Some columns have only a few such values; for others it is most rows (e.g. `Total Asset Growth Rate`: 6,017 of 6,819). These values look like an encoding artifact. A decision tree only depends on the order of the values, so it is unaffected. A linear selector (LASSO) is affected, so its treatment is decided in Step 2 and fitted inside the cross-validation folds.

3. Notebook structure (applies to the whole notebook)

```
# Taiwanese Bankruptcy Prediction        ← title, dataset citation, 4-step overview
## 0. Setup                              ← imports, constants (SEED, C_FN, C_FP, THRESHOLD)
## 1. Problem Formulation & EDA
### 1.1 Problem formulation
### 1.2 Data loading and integrity checks
### 1.3 Train/test split
### 1.4 Exploratory analysis (training set)
### 1.5 Step 1 takeaways
## 2. Variable Selection                 ← same sub-structure, defined in Step 2's plan
...
```

- Decisions and assumptions are blockquotes: `> **Decision:** …` or `> **Assumption:** …`.
- Findings are one-line `**Observation:** …` cells placed after the output they describe.
- Dataset citation: UCI ML Repository, *Taiwanese Bankruptcy Prediction*, DOI 10.24432/C5004D. Original study: Liang et al. (2016), *European Journal of Operational Research* 252(2).

4. Planned code cells (one task each)

#	Cell
1	Imports
2	Constants: `SEED`, `C_FN`, `C_FP`, `THRESHOLD`
3	Load the CSV and strip the leading spaces from column names
4	Shape and first rows
5	Missing values and duplicate rows
6	Target counts and proportions
7	Bar plot of the target
8	Find the constant columns, then drop them
9	Split into X and y, then stratified train/test split
10	Check the bankrupt rate in train and test
11	Scale check: columns with values > 1 (count per column)
12	Redundancy: number of feature pairs with |r| > 0.95, plus the top pairs
13	Univariate signal: ROC-AUC of each feature alone, top 10
14	Boxplots of the 4 strongest features by class

What I expect to find, from a first look: 23 feature pairs with |r| > 0.95, several of them exact duplicates (r = 1.00). Profitability ratios (EPS, ROA, net income / assets) separate the classes best on their own, with AUC around 0.86–0.89. Both points motivate Step 2.

5. Points for you to confirm or change

- **Cost ratio C_FN : C_FP = 10 : 1.** Alternatives: about 30 : 1, which roughly corresponds to balancing the two classes, or a non-cost metric such as F1 or recall at a fixed precision.
- **Test size of 20%.** Moving to 25% gives about 55 bankrupt firms in the test set instead of 44.
- **EDA on the full dataset.

Environment

The project has a `.venv` created with `uv` (Python 3.12), and the dependencies are pinned in `requirements.txt`. The notebook will be saved with its outputs, at `notebooks/bankruptcy_prediction.ipynb`.

`AgentDiscussion/step1/step1_recap.md` Step 1 recap: written by the LLM, reviewed by the authors

Step 1 — Problem Formulation & EDA (recap)

Notebook: `notebooks/bankruptcy_prediction.ipynb`, sections 0 and 1. It runs end to end on the `.venv` kernel and is saved with its outputs.

What was built

Section	Content
Title	Short introduction, dataset citation, outline of the 4 steps
0. Setup	Imports; constants `SEED = 42`, `C_FN = 10`, `C_FP = 1`, `THRESHOLD = 1/11`
1.1 Problem formulation	Target, inputs, decision, loss table, Bayes rule, metric, baseline, plus 2 assumption cells (costs; no leakage and independent firms)
1.2 Loading and integrity	Load and clean column names; shape; missing values and duplicates; class counts and bar plot; drop the constant column; separate X and y
1.3 Exploratory analysis	Values above 1; Spearman correlation matrix (heatmap) and correlated pairs; AUC of each feature alone; boxplots of the top 4 features
1.4 Train/test split	Stratified 80/20 split and a check of both sets
1.5 Takeaways	5 bullets that summarise the step

There are 18 code cells and 7 blockquote cells (decisions and assumptions), plus one `**Observation:**` line after each output that needs interpreting.

Key results

- **Data quality:** 6,819 firms, no missing values, no duplicate rows. `Net Income Flag` has the same value everywhere and is dropped, which leaves **94 features**.
- **Class imbalance:** 220 bankrupt firms (3.2%). The "never flag" baseline costs about **0.32 per firm**.
- **Extreme values:** 24 features mix values in $[0, 1]$ with values between 6e5 and 1e10, with nothing in between. A few firms are affected in some features; 6,017 firms are affected in `Total Asset Growth Rate`.
- **Redundancy:** 50 pairs of features have Spearman $|\rho| > 0.95$. These pairs involve 47 of the 94 features, and 13 of the pairs have $|\rho| > 0.999$. The heatmap shows blocks of related ratios: profitability, per-share values, growth, leverage and cash flow.
- **Signal:** the strongest single predictors are profitability ratios (EPS, net income / assets, ROA), with AUC between 0.86 and 0.89. Bankrupt firms clearly score lower, but the two classes still overlap.
- **Split:** 5,455 firms for training (176 bankrupt) and 1,364 for testing (44 bankrupt).

Differences from the plan

1. **Exploration on the full dataset.** I followed your edit in section 5 of the plan. Two places in the plan still say "training set only": §2 item 4 and the §3 outline. I treated your section 5 edit as the decision.
2. **Section order is now 1.3 EDA → 1.4 split** (the plan had split → EDA). Since the exploration uses the full data, splitting before it would serve no purpose, and the notebook reads more naturally this way. Tell me if you want the original order back.
3. **The "find constant columns, then drop them" step is two cells**, with the decision cell in between. This follows the "one task per cell" rule.
4. **Separating X and y has its own cell at the end of 1.2**, so the exploration and the split share the same `X` and `y`.
5. **The boxplots hide outliers** (`showfliers=False`, and the plot title says so). With outliers shown, the boxes were unreadable.

Changes after your review

- **Added a correlation matrix** (heatmap of all 94 features, in column order) to section 1.3, just before the list of correlated pairs.
- **Switched from Pearson to Spearman correlation**, for the heatmap and for the list of pairs. A decision cell in the notebook records this.
  - Why: Pearson missed ratios that are inverses of each other, such as `Current Ratio` and `Current Liability to Current Assets` ($\rho = 0.999$), and it was distorted by the extreme values.
  - Effect: the count of redundant pairs went from 23 to 50.
  - Spearman measures order, which is all a tree uses.
  - To go back to Pearson, change `method="spearman"` in a single cell.
- **Wrote the math in LaTeX** (`$...$`) in every markdown cell that contains math. In 1.1, the decision rule and the cost metric are now displayed equations, and the rule includes its one-line derivation.
- **Corrected the cell counts** in this recap. The first version said 21 code cells and 7 decision cells; the real numbers were 16 and 6.

Carried into Step 2

- The 24 features with extreme values need treatment before any linear or LASSO selector. That treatment must be fitted inside the cross-validation folds.
- Selection and tuning only see `X_train` and `y_train`. `X_test` and `y_test` are not touched until Step 4.
- New imports will be added to the single Setup cell rather than scattered through the notebook.

`AgentDiscussion/step2/step2_plan.md` Step 2 plan: written by the LLM, reviewed and edited by the authors

Step 2 — Variable Selection (plan)

Everything in this step uses the training set only (`X_train`, `y_train`).

1. Method: LASSO logistic regression

The outcome is binary, so the LASSO of Lecture 4 becomes an **L1-penalised logistic regression**, with an unpenalised intercept:

$$\min_{b_0,\,\beta} \; -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \log p_i + (1-y_i)\log(1-p_i)\Big] + \lambda\|\beta\|_1, \qquad p_i = \frac{1}{1+e^{-(b_0 + x_i^{\top}\beta)}}$$

In scikit-learn the penalty is expressed as `C` $= 1/(n\lambda)$: a smaller `C` means a stronger penalty and fewer variables.

Why LASSO rather than forward selection (Lecture 3.2):

- One parameter controls the number of variables.
- It copes with the 50 highly correlated pairs found in Step 1.
- One fit takes about 0.5 s, so it can be refit inside every cross-validation fold of Step 3, as the brief requires.
- Forward selection is greedy and would need thousands of model fits per selection (up to 94 candidates per step), repeated again in every fold.

Role in the pipeline: the LASSO only *chooses* the variables. The decision tree in Step 3 receives those variables with their **raw values**, so its split thresholds stay in the original ratio units.

2. Decisions (each one becomes a short markdown cell)

1. **The LASSO input is transformed with a rank-based quantile transform** (to a standard normal), fitted inside each fold. This transform is not passed on to the tree.
   - Why not cap the extreme values at 1: they carry signal. Within the extreme-value group of `Total Asset Growth Rate`, the magnitude alone gives an AUC of 0.72. Capping would erase that.
   - What the transform does: it keeps the order of the values, removes the magnitude problem, and puts all features on the same scale, which a fair L1 penalty needs.
   - Evidence, 5-fold CV log-loss on the training set:

Transform	CV log-loss
Quantile	**0.085**
Cap at 1 + standardise	0.098
log + standardise	0.100
Standardise only	0.100

2. **The intercept stays unpenalised, as in the course.**
   - The problem: scikit-learn's `liblinear` solver penalises the intercept by default, which visibly distorts the path at strong penalties.
   - The fix: setting `intercept_scaling=100` makes that penalty negligible.
   - Check: the `saga` solver, which never penalises the intercept, gives the same path but runs about 6× slower and raises a convergence warning.
3. **No class weights in the selector.** It is a penalised maximum-likelihood fit. The misclassification costs enter in Step 3, where the decision is made.
4. **`C` is chosen by 5-fold stratified cross-validation, minimising log-loss.**
   - Why log-loss: it is a proper scoring rule for probabilities, the classification counterpart of the validation MSE used in Lectures 3–4.
   - Grid: 13 values of `C` from 10^{-3} to 1, evenly spaced on a log scale.
   - The same CV splitter (`StratifiedKFold(5, shuffle=True, random_state=SEED)`) is reused in Step 3.
5. **Selection stability is measured with 100 stratified bootstrap resamples** of the training set (Lecture 4). `C` stays fixed and the full procedure is refit on each resample. The resulting selection frequencies are **descriptive** and do not change the selected set.
6. **Hand-off to Step 3:** the selection procedure is wrapped as a scikit-learn `SelectFromModel` object. It sits in front of the tree and is **refit inside every CV fold**, with `C` fixed at the value chosen here.
   - Step 3 also compares this fixed `C` with tuning `C` together with the tree, on the decision cost.
   - Both versions use the same folds and the same metric.
   - The joint version is preferred only if its CV cost is lower by more than one standard error across folds.

3. Notebook structure

```
## 2. Variable Selection
### 2.1 Method                    ← objective, why LASSO, decision cells 1–3
### 2.2 Choice of the penalty     ← CV over C, path plot
### 2.3 Selected variables        ← coefficients at the chosen C
### 2.4 Selection stability       ← bootstrap frequencies
### 2.5 Step 2 takeaways
```

4. Planned code cells (one task each)

New imports go into the existing Setup cell.

#	Cell
1	Define the 5-fold stratified CV splitter (reused in Step 3)
2	Define the LASSO pipeline: quantile transform, then L1 logistic regression
3	Cross-validate the grid of `C` values (log-loss)
4	Count the selected variables for each `C` (refit on the full training set)
5	Plot CV log-loss and the number of selected variables against `C`, with the chosen `C` marked
6	Fit the LASSO at the chosen `C`; list the selected variables with their coefficients
7	Bootstrap: selection frequency of each variable over 100 resamples
8	Bar chart of the selection frequencies
9	Build the selector for Step 3 (`SelectFromModel` wrapping the LASSO pipeline at the chosen `C`)

Preview from my checks:

- **Chosen penalty:** the best `C` is about 0.18. It keeps **43 of the 94 variables**, with a CV log-loss of 0.085 against about 0.14 for the intercept-only model.
- **Stability:** of the selected variables, only about 18 appear in at least 80% of resamples, and the median selection frequency is 0.65. Correlated ratios swap places from one resample to the next, which is the instability Lecture 4 warns about.
- **Runtime:** Step 2 adds about 2 minutes to the notebook (40 s for the path, about 65 s for the bootstrap).

5. Points for you to confirm or change

- **Rule for choosing `C`: decided.** We use the lowest CV log-loss (course protocol). Step 3 compares it with tuning `C` together with the tree (see decision 6). The one-standard-error rule was dropped because its result depends on the solver.
- **Number of bootstrap resamples:** 100, which takes about 1 minute.

`AgentDiscussion/step2/step2_recap.md` Step 2 recap: written by the LLM, reviewed by the authors

Step 2 — Variable Selection (recap)

Notebook: section 2 of `notebooks/bankruptcy_prediction.ipynb`. The whole notebook runs end to end in about 1.5 minutes and is saved with its outputs. Step 2 uses `X_train` and `y_train` only.

What was built

Section	Content
Setup	New imports added to your imports cell; your `ggplot` style is kept
2.1 Method	The LASSO objective (LaTeX); decision LASSO vs forward selection; evidence that the extreme values carry signal; decisions on the quantile transform, the unpenalised intercept and the absence of class weights; definition of the `lasso` pipeline
2.2 Choice of the penalty	Decision on CV log-loss; the `cv` splitter; cross-validation over a 13-value grid of `C`; number of variables per `C`; a 2-panel plot of the path
2.3 Selected variables	The 43 coefficients at the chosen `C`
2.4 Selection stability	Decision on the bootstrap; 100 stratified resamples; bar chart of the frequencies; decision on the hand-off; the `selector` object plus a check cell
2.5 Takeaways	5 bullets

Section 2 adds 11 code cells and 7 decision cells.

Key results

- **Extreme values carry signal.** Among the 4,816 training firms whose `Total Asset Growth Rate` exceeds 1, the value alone gives an AUC of 0.72 (0.28 with the direction flipped). This confirms that capping them at 1 would lose information.
- **Chosen penalty: C ≈ 0.178**, which keeps **43 of the 94 variables**. Its CV log-loss is 0.0855, against 0.143 with no variable. The curve is flat: every value from 21 to 59 variables is within one standard error (0.002) of the best.
- **What was selected:** leverage, profitability, liquidity and activity ratios. For the leading variables the signs make sense: more debt raises the risk, and more profit or liquidity lowers it.
- **Stability is moderate.**
  - Only 15 of the 43 selected variables appear in at least 80% of resamples. The median frequency is 0.69 and the lowest is 0.25.
  - `Research and development expense rate` is not selected on the full training set, yet appears in 81% of resamples.
  - The LASSO keeps both members of a practically identical pair (`Current Liabilities/Liability` and `Current Liability to Liability`), each with a tiny coefficient.
  - The variable list is a predictive shortlist, not a set of causes.
- **Check passed:** `selector`, fitted on the full training set, keeps exactly the 43 variables listed in 2.3.

Differences from the plan

1. **Extra evidence cell in 2.1** (the AUC within the extreme-value group). It backs up the quantile-transform decision inside the notebook. The comparison of the four transforms (0.085 vs 0.098–0.100) is *cited* from my preliminary check rather than re-run, because re-running it would add about 6 minutes to the notebook.
2. **"LASSO rather than forward selection" is its own decision cell**, instead of a paragraph inside the method text.
3. **Extra check cell after the selector**, to confirm it reproduces the 43 variables.
4. **The cross-validation over `C` runs in parallel** (`n_jobs=-1`). This brings the whole notebook down to about 1.5 minutes.

5. **The stability chart shows all 92 variables that were selected at least once.** It is tall. If you want a more compact figure for the report, I can limit it to the top 40.

Changes after your review

- **The stability chart has a proper legend.** Blue means "in the 43-variable shortlist" and grey means "not in the shortlist". A dashed line marks 80% of resamples, the boundary for robust selection.
- **The chart title is now "Bootstrap selection frequency (100 resamples)".** This needed `from matplotlib.patches import Patch` in the Setup cell.
- **The observation cell points to the dashed line.**
- **Your explanation cell under the chart is unchanged.**

Carried into Step 3

- `cv` (the 5 stratified folds) and `selector` (unfitted; it is refit inside every fold) are ready.
- The comparison of fixed `C` against `C` tuned together with the tree, using the decision rule agreed on: prefer the joint version only if its CV cost is lower by more than one standard error.
- The misclassification costs have to enter the tree, through class weights or the threshold of 1/11. That is a decision for the Step 3 plan.

`AgentDiscussion/step3/step3_plan.md` Step 3 plan: written by the LLM, reviewed and edited by the authors

Step 3 — Model Selection and Decision Tree Tuning (plan)

Everything in this step uses the training set only, with the 5 folds `cv` from Step 2. The test set is first used in Step 4.

1. Decision rule and metric

- **The tree estimates probabilities and the loss table decides (Lectures 1 and 3).**
  - A standard Gini tree is grown.
  - Each leaf estimates $\hat{\eta}(x) = P(Y = 1 \mid x)$ by its share of bankrupt firms.
  - A firm is flagged when $\hat{\eta}(x) \geq 1/11$, the Bayes threshold from 1.1.
  - In code, scikit-learn's `FixedThresholdClassifier(..., threshold=THRESHOLD)` wraps the model, so `predict()` returns the cost-optimal decision directly.
- **The metric is the average cost per firm**, $(C_{FN} \cdot \text{FN} + C_{FP} \cdot \text{FP})/n$. It is turned into a scikit-learn scorer and used for all tuning and comparisons.

2. Decisions (each one becomes a short markdown cell)

1. **The costs enter through the threshold, not through class weights.**
   - The alternative: grow the tree with cost-weighted Gini (`class_weight={0: 1, 1: 10}`).
   - What I found: that gives the same CV cost within one standard error (0.162 vs 0.159 in my check).
   - Why the threshold: it keeps the course's reading of leaves as probability estimates. It also lets Step 4 re-run the cost-ratio sensitivity by changing only the threshold, with no retraining.
2. **Tuning grid (Gini criterion, `random_state = SEED`):**

Parameter	Values
`max_depth`	2, 3, 4, 5, 6, 8, None (no limit)
`min_samples_leaf`	1, 10, 20, 40, 80, 120, 160, 200

Why the leaf sizes go so high:

- Leaf shares are only trusted as probabilities if leaves are large enough (Lecture 3: "small leaves make probability estimates unstable").
- With a 3.2% bankruptcy rate, a leaf of 120 firms contains only about 4 bankruptcies on average.
- My first grid stopped at 80, and that edge value won, so I extended the grid.

3. **Four candidates, all cross-validated on the same `cv` folds.** The selection step is refit inside every fold.

	Candidate	Tuned
M0	Baseline: never flag	–
M1	Tree on all 94 features	depth, leaf
M2	LASSO selector (`C` fixed at 0.178, from Step 2) + tree	depth, leaf
M3	LASSO selector + tree, with `C` tuned jointly	`C`, depth, leaf

For M3, `C` takes the 7 values of Step 2's grid from 0.01 to 0.316. That range includes 0.178, so M3 contains M2 as one of its options.

4. **Selection rule (as agreed in Step 2):** M2, the course protocol, is the reference. An alternative replaces it only if its CV cost is lower by more than one standard error. Standard error = standard deviation across the 5 folds / $\sqrt{5}$.

5. **The final model** is the chosen candidate refit on the full training set with its best parameters. It is then **frozen** for Step 4.

3. Notebook structure

```
## 3. Model Selection and Decision Tree Tuning
### 3.1 Decision rule and metric        ← cost function, scorer
### 3.2 Candidates and tuning grid      ← decision cells 1–4, grid
### 3.3 Cross-validated tuning          ← M0, M1, M2, M3, validation curve
### 3.4 Model comparison                ← table, plot, selection rule
### 3.5 Final model                     ← frozen pipeline, what it uses
### 3.6 Step 3 takeaways
```

4. Planned code cells (one task each)

New imports go into the Setup cell: `DecisionTreeClassifier`, `FixedThresholdClassifier`, `DummyClassifier`, `Pipeline`, `make_scorer`, `confusion_matrix`, `cross_val_score`.

#	Cell
1	Cost function: average misclassification cost per firm
2	Scorer built from the cost function
3	Tree grid (`max_depth`, `min_samples_leaf`)
4	M0: CV cost of "never flag"
5	M1: grid search, thresholded tree on all features
6	M2: grid search, thresholded pipeline (selector with fixed `C`, then tree)
7	M3: same as M2, with `C` added to the grid
8	Validation curve for M2: training and CV cost against depth, for leaf size 1 and for the chosen leaf size
9	Comparison table: CV cost ± standard error, number of variables, best parameters (M0–M3)
10	Bar chart of the comparison with ±1 standard-error bars
11	Apply the selection rule and keep the final model
12	Describe the final model: parameters, variables selected, variables the tree actually uses, number of leaves

5. Preview from my checks (same folds and seed, so the notebook should reproduce them)

Candidate	CV cost per firm	Best parameters
M0 never flag	0.323 ± 0.002	–
M1 all features	0.163 ± 0.009	depth 4, leaf 120
M2 LASSO, fixed `C`	**0.151 ± 0.008**	depth 4, leaf 120 (43 selected; the tree uses 8)
M3 LASSO, joint `C`	0.148 ± 0.009	`C` = 0.01 (about 7 variables), depth 3, leaf 10

- **All trees roughly halve the baseline cost.**
- **Selection helps:** M2 beats M1 by 0.012, which is more than one standard error.
- **M3 does not beat M2 by enough:** its gain of 0.003 is well below one standard error, so the rule keeps **M2**.
  - Interesting side result: tuning `C` on the decision cost picks a much sparser selection (about 7 variables) at practically the same cost.
  - In practice the final M2 tree uses only 8 variables anyway, so the two models are similarly interpretable.
- **The validation curve tells the overfitting story (Lecture 2):** with leaves of size 1, training cost falls to 0 as depth grows, while CV cost rises from 0.17 to 0.23. With leaves of 120, depth 4 is best.
- **Runtime:** Step 3 adds about 2.5 minutes, almost all of it for M3.

6. Points for you to confirm or change

Decided (you approved the recommendations):

- The costs enter through the threshold.
- The grid stays as listed in decision 2.
- The agreed rule stays as it is, with no tie-breaker for fewer variables.

The original points are kept below for reference.

- **How costs enter:** through the threshold (recommended), or through class weights.
- **The grid in decision 2.**
- **Keeping the agreed rule, even though M3 reaches the same cost with about 7 variables.** The alternative is to add a tie-breaker: "within one standard error, prefer fewer selected variables", which would pick M3. I recommend keeping the agreed rule. It was set before seeing these results, and changing a rule after seeing the results weakens it.

`AgentDiscussion/step3/step3_recap.md` Step 3 recap: written by the LLM, reviewed by the authors

Step 3 — Model Selection and Decision Tree Tuning (recap)

Notebook: section 3 of `notebooks/bankruptcy_prediction.ipynb`. The whole notebook runs end to end in about 4 minutes; tuning M3 takes about 2 of them. It is saved with its outputs, and Step 3 uses the training set only.

What was built

Section	Content
Setup	New imports added to the single imports cell
3.1 Decision rule and metric	Leaf share as $\hat{\eta}(x)$, flag when ≥ 1/11, empirical-risk formula (LaTeX); decision on threshold vs class weights; `avg_cost` and `cost_scorer`
3.2 Candidates and grid	Decisions on the grid, the 4 candidates and the selection rule; `tree_grid`
3.3 Cross-validated tuning	M0, M1, M2, M3, each with its own cell; M2 validation curve (2 panels)
3.4 Model comparison	Comparison table, bar chart with ±1 standard-error bars, rule table, decision cell
3.5 Final model	`final_model` frozen; description (variables selected and used, depth, leaves)
3.6 Takeaways	6 bullets

Section 3 adds 13 code cells and 5 decision cells.

Key results (identical to the plan's preview)

Candidate	CV cost per firm	SE	Depth	Leaf	C	Variables
M0 never flag	0.3226	0.0016	–	–	–	0
M1 all features	0.1630	0.0090	4	120	–	94
M2 LASSO, C fixed	**0.1512**	0.0077	4	120	0.178	43
M3 LASSO, C joint	0.1483	0.0087	3	10	0.01	7

- **M2 is kept by the agreed rule.**
  - M3's gain is 0.003, well below its standard error of 0.009.
  - M1 is worse by 0.012, more than one standard error, so the variable selection helps.
- **Final model:** the LASSO keeps 43 variables, followed by a depth-4 tree with **9 leaves that uses only 8 variables**. Those 8 are profitability (EPS, continuous interest rate), debt and its cost (borrowing dependency, interest-bearing debt rate, interest coverage), growth (net value growth) and activity (inventory/working capital, quick asset turnover).
- **Validation curve:**
  - With leaves of 1 firm, training cost falls to 0 as depth grows, while CV cost rises from 0.17 to 0.23. That is clear overfitting.
  - With leaves of at least 120 firms, the gap between training and CV cost stays small, and depth 4 is best.

Differences from the plan

1. **M2's grid states `C` explicitly** (`[best_C]`) instead of relying on the value preset inside `selector`. This is more robust and makes M2 and M3 directly parallel, since M3 just swaps in `C_grid[4:11]`.
2. **"Apply the rule and keep the final model" became two cells** (the rule table, then the freeze), to keep one task per cell. That is why there are 13 code cells instead of 12.
3. **Section 3.1 shows the metric as an empirical-risk formula**, and `avg_cost` computes exactly that: the mean of the per-firm losses.
4. **Two pandas 3 adjustments:**
   - In the validation-curve plot, depth "None" is converted to the label "no limit" before plotting.
   - The comparison table uses `convert_dtypes()`, so depths and leaf sizes display as whole numbers.

Carried into Step 4

- **What Step 4 uses:**
  - `final_model`, frozen: a `FixedThresholdClassifier` whose `predict()` returns the flag decision.
  - `avg_cost`.
  - The never-flag baseline, for comparison on the test set.
- **First use of the test set:**
  - test cost compared with the baseline
  - confusion matrix, recall and precision
  - drawing the tree for interpretation
- **Cost-ratio sensitivity (5 and 20):** `final_model.set_params(threshold=...)` changes the threshold (1/6, 1/21) **without refitting**.
- **Limitations to discuss:**
  - no time-based split
  - the costs are stipulated
  - CV noise (SE about 0.008, with only about 35 bankruptcies per fold)
  - no nested CV for the M2 vs M3 comparison
  - moderate stability of the LASSO selection

`AgentDiscussion/step4/step4_plan.md` Step 4 plan: written by the LLM, reviewed and edited by the authors

Step 4 — Evaluation of the Final Model (plan)

This step is the **first and only use of the test set** (1,364 firms, 44 bankrupt). Following Lecture 2, the frozen final model from Step 3 is evaluated once, and nothing is tuned or re-chosen afterwards. **I wrote this plan without computing any test-set result.** That way, the choice of what to report cannot be influenced by the results. The previews below come from the training set only.

1. Decisions (each one becomes a short markdown cell)

1. **Only the final model (M2) and the "never flag" baseline are scored on the test set.** M1 and M3 are not.
   - Reporting the test scores of rejected candidates invites choosing again on the test set, which would make the test score part of model selection (Lecture 2).
2. **The test results are reported as:**
   - the average cost per firm (primary metric)
   - recall, precision and the number of flagged firms
   - the confusion matrix
   - accuracy, shown only to illustrate why it was rejected in 1.1
3. **Uncertainty: a bootstrap confidence interval, because Lecture 2's Hoeffding bound is too loose here.**
   - The loss lies in $[0, 10]$ and there are 1,364 test firms. Hoeffding then gives a margin of **±0.37** at 95%. That is larger than the gap we want to measure (≈ 0.32 vs ≈ 0.15), so the bound is valid but uninformative (Lecture 2: "bounds larger than the loss range are valid but uninformative"). It is shown in one line.
   - Instead: a **95% percentile bootstrap** with 1,000 resamples of the test firms, keeping the predictions fixed. The interval is computed for the model's cost, the baseline's cost and their **paired difference**.
4. **Sensitivity to the cost ratio (announced in 1.1):** $C_{FN}/C_{FP} \in \{5, 10, 20\}$.
   - For each ratio r, the decision threshold becomes $1/(1+r)$ (Bayes rule). The tree is **not retrained**.
   - Each version is compared with the baseline under the same ratio.
   - In code, the probabilities are computed once (`predict_proba`) and thresholded per ratio, so `final_model` is never modified.
5. **Switching points:** for each leaf, the cost ratio at which its decision flips, $r^* = (1-\hat{\eta})/\hat{\eta}$, computed from its training share $\hat{\eta}$. This explains the sensitivity results directly from the loss table (Lecture 1, "different costs can reverse the decision").
6. **Interpretation:**
   - **Tree diagram.** The final tree is drawn with `plot_tree`, in raw ratio units, with node ids.
   - **Leaf table.** Each leaf gets its training firms, its bankruptcy share and its decision (flag if the share is ≥ 1/11).
   - **Why the table is needed.** `plot_tree` labels each leaf with its *majority* class, which is "healthy" for all 9 leaves. That label would mislead, so the decision comes from the table instead.
7. **The notebook closes with a decision memo** following Lecture 1's structure: the action and the information used, the probability and loss assumptions, the comparison with a feasible alternative, and the limitations and conditions that would change the choice.

2. One small change to Step 3 code

`avg_cost` gets optional cost arguments: `def avg_cost(y_true, y_pred, c_fn=C_FN, c_fp=C_FP)`.

- Step 3 is unaffected, because the defaults are the same values.
- The sensitivity analysis can then reuse the function instead of repeating the formula.

3. Notebook structure

```
## 4. Evaluation of the Final Model
### 4.1 Test-set performance          ← decisions 1–2, metrics table, confusion matrix
### 4.2 Uncertainty of the test estimate ← Hoeffding (1 line), bootstrap intervals
### 4.3 Sensitivity to the cost ratio  ← ratios 5/10/20, switching points
### 4.4 Interpretation of the final tree ← tree diagram, leaf table
### 4.5 Recommendation and limitations ← decision memo (markdown)
```

4. Planned code cells (one task each)

New imports go into the Setup cell: `recall_score`, `precision_score`, `accuracy_score`, `ConfusionMatrixDisplay`, `plot_tree`.

#	Cell
1	Helper `test_metrics(y_true, y_pred, c_fn)`: cost, recall, precision, accuracy, flagged firms
2	Test predictions of the final model and of the baseline (flags nobody)
3	Metrics table: final model vs baseline
4	Confusion matrix of the final model (plot)
5	Hoeffding half-width for the test cost (one line)
6	Bootstrap: 1,000 resamples of the test firms; costs of model, baseline and difference
7	95% percentile intervals table
8	Test probabilities of the final model (computed once)
9	Sensitivity table for ratios 5, 10, 20: threshold, model cost, baseline cost, recall, precision
10	Leaf table: training firms, bankruptcy share, decision at 1/11, switching ratio r^*
11	Plot of the final tree (node ids, proportions, raw units)

5. Preview from the training set only

Leaf share $\hat{\eta}$	Training firms	Flagged at 1/6 (r = 5)	1/11 (r = 10)	1/21 (r = 20)	r^*
0.420	150	✓	✓	✓	1.4
0.397	121	✓	✓	✓	1.5
0.116	146	–	✓	✓	7.6
0.093	129	–	✓	✓	9.8
0.093	129	–	✓	✓	9.8
0.039	155	–	–	–	24.8
0.016, 0.015, 0.003	122, 264, 4,239	–	–	–	> 60

- **Ratios 10 and 20 give identical decisions.** Every ratio between about 9.8 and 24.8 does, so the recommendation is robust to doubling the cost of a missed bankruptcy.

- **Ratio 5 flags only the two high-risk leaves:** 271 training firms instead of 675.
- **Two leaves (258 firms) are flagged only narrowly.** Their share of 0.093 is barely above 1/11, so with a cost ratio below 9.8 they would not be flagged. This goes into the limitations.
- **Runtime:** Step 4 adds only a few seconds.

6. Points for you to confirm or change

- **Score only M2 and the baseline on the test set: decided.**
- **Bootstrap: decided.** 1,000 resamples of the test firms, not stratified, so that the share of bankrupt firms also varies.
- **Switching-point column (r^*) in the leaf table: decided.** Kept, as an addition to the original Step 1 plan, which only mentioned ratios 5 and 20.

`AgentDiscussion/step4/step4_recap.md` Step 4 recap: written by the LLM, reviewed by the authors

Step 4 — Evaluation of the Final Model (recap)

Notebook: section 4 of `src/bankruptcy_prediction.ipynb`, which you moved from `notebooks/`. I applied the edits there and kept your `.venv` kernel selection. The full notebook runs end to end in about 4 minutes with no errors and is saved with its outputs.
The test set was used exactly once, on the frozen final model (M2) and the "never flag" baseline only.

What was built

Section	Content
Setup	New imports: `recall_score`, `precision_score`, `accuracy_score`, `ConfusionMatrixDisplay`, `plot_tree`
Step 3	`avg_cost` gained optional `c_fn`/`c_fp` arguments, with the same defaults, so Step 3 results are unchanged (M2 still 0.1512)
4.1 Test-set performance	2 decision cells; `test_metrics` helper; predictions; metrics table; confusion matrix
4.2 Uncertainty	Decision; Hoeffding half-width; 1,000-resample bootstrap; 95% intervals
4.3 Sensitivity	Decision; test probabilities computed once; table for ratios 5/10/20; decision on switching points; leaf table with r^*
4.4 Interpretation	Decision (why no class labels); tree diagram with node ids
4.5 Recommendation and limitations	Decision memo in Lecture 1's structure; 7 limitations

Section 4 adds 11 code cells and 6 decision cells. The notebook now has 132 cells in total.

Key results

Test set (1,364 firms, 44 bankrupt)	Final model	Baseline
Average cost per firm	**0.179**	0.323
95% bootstrap interval	[0.134, 0.233]	[0.235, 0.418]
Recall	72.7% (32 of 44)	0%
Precision	20.5%	–
Flagged firms	156 (about 1 in 9)	0
Accuracy	90.0%	96.8%

- **The model cuts the cost by 45%.** The paired interval for the gain, [−0.228, −0.065], excludes 0.
- **The test cost (0.179) is higher than the CV cost (0.151),** but the CV value lies inside the test interval. The gap is consistent with the noise you'd expect from only 44 test bankruptcies.
- **Hoeffding's margin is ±0.37:** valid, but too wide to separate the two rules. This is what motivated using the bootstrap.
- **Sensitivity:** the model beats the baseline at every cost ratio, cutting the cost by 35% at r = 5, 45% at r = 10 and 59% at r = 20. Ratios 10 and 20 give identical decisions, and ratio 5 flags 68 firms.
- **Switching points:**
  - Nodes 1 and 13 are flagged for any ratio above 1.5.
  - Node 12 switches at 7.6.
  - Nodes 6 and 15 switch at 9.75, so they are flagged only narrowly.
  - No leaf switches between 9.75 and 24.8.
- **Interpretation.** The tree raises an alarm in three situations:
  - the lowest net value growth (42% bankrupt);
  - high borrowing combined with weak profitability, especially when the debt is expensive (40% and 12%);
  - two smaller pockets at about 9%.

  Node 9 holds 78% of the firms and has a bankruptcy share of only 0.3%.

Differences from the plan

1. **The leaf table sits at the end of 4.3,** where it explains the sensitivity results, rather than in 4.4. Section 4.4 refers back to it to read the tree diagram.
2. **The tree diagram uses `precision=5`.** With 3 decimals, splits such as `Net Value Growth Rate <= 0.00037` displayed as `<= 0.0`. Colours and class labels are left out, as planned.
3. **Both results tables have one row per model or per ratio,** so counts display as whole numbers (156, not 156.0000).
4. **The notebook's location changed** (`notebooks/` → `src/`) partway through this step. The first run had already been written before the move, so I re-applied the final edits to `src/`.

Notes

- `step4_plan.md` is still at the root of `AgentDiscussion/`. I put this recap in `AgentDiscussion/step4/` to match your new layout, and I left the plan where it is in case you have it open. Move it whenever you like.
- The recap files for Steps 1–3 still mention `notebooks/` as the notebook's location. I left them unchanged, since they record what was true at the time.